

Web Application Security Monitoring Using Apache, ModSecurity (OWASP CRS), and Wazuh SIEM

Project Overview

Web applications are one of the most common attack surfaces in modern infrastructure. Attacks such as **SQL Injection**, **Cross-Site Scripting (XSS)**, **directory traversal**, and **automated scanning** frequently target exposed web servers.

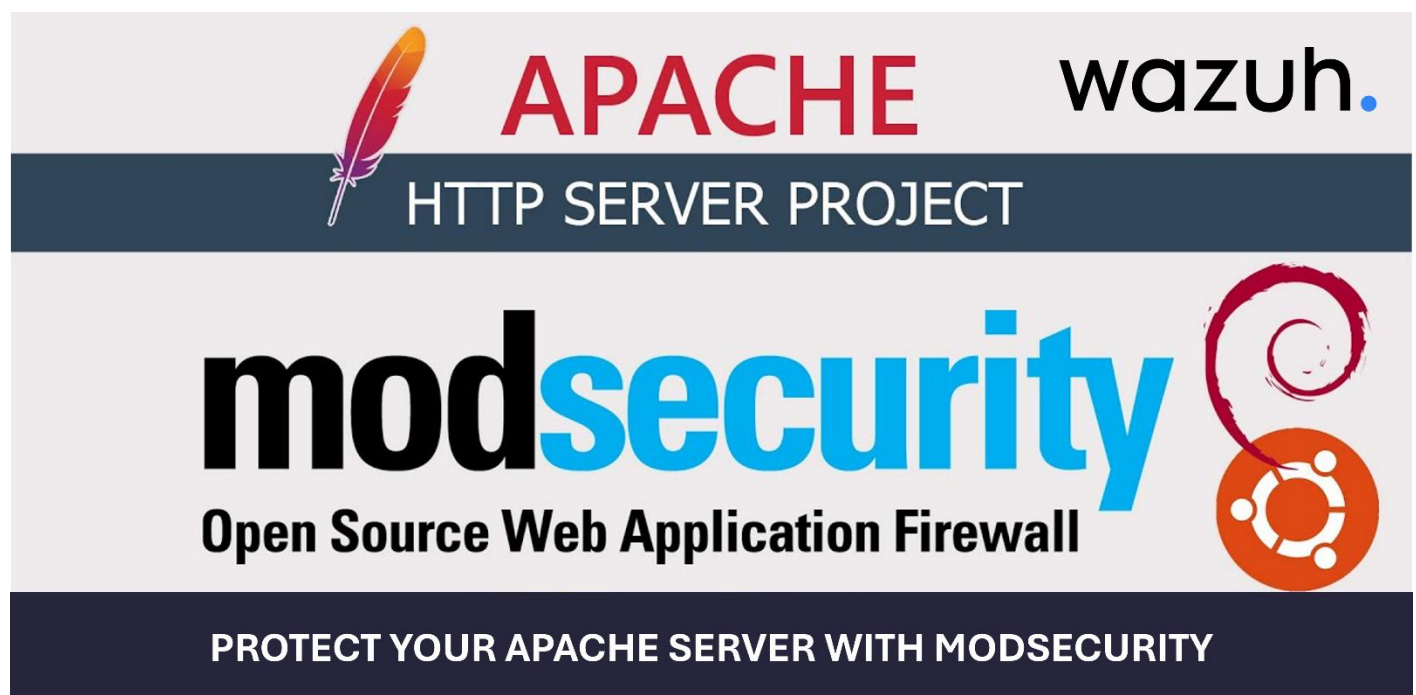
In this project, an **Apache web server** is secured using **ModSecurity**, a Web Application Firewall (WAF), along with the **OWASP Core Rule Set (CRS)** to detect and block malicious HTTP requests.

All ModSecurity security events and audit logs are then **integrated with Wazuh SIEM** to enable centralized monitoring, alerting, and threat visibility.

This setup simulates a **real-world SOC use case**, where web application attacks are detected at the WAF layer and monitored centrally through a SIEM platform.

Project Objectives

- Deploy Apache web server on Ubuntu
- Enable ModSecurity as a Web Application Firewall
- Load OWASP Core Rule Set (CRS)
- Generate real attack traffic (SQLi, scanning, etc.)
- Collect ModSecurity logs using Wazuh Agent
- Visualize and analyze web attack alerts in Wazuh Dashboard



The banner features the Apache logo (a feather) and the text "APACHE" in red, followed by "wazuh." in black. Below this, a dark blue bar contains the text "HTTP SERVER PROJECT" in white. The main section of the banner is light gray and contains the "modsecurity" logo in black and blue, with the tagline "Open Source Web Application Firewall" in black. To the right of the ModSecurity logo is the Wazuh logo, which consists of a red spiral and an orange circle with white dots. At the bottom, a dark blue bar contains the text "PROTECT YOUR APACHE SERVER WITH MODSECURITY" in white.

Tools & Technologies Used

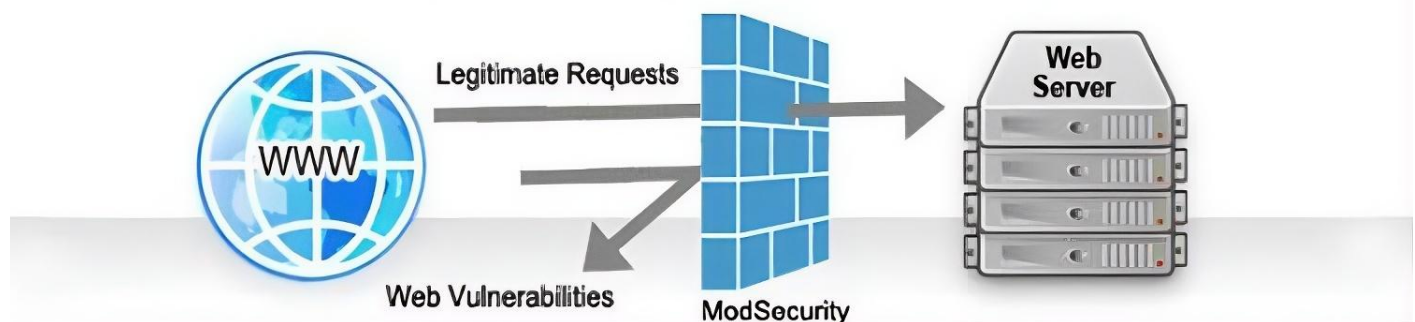
TOOL	PURPOSE
UBUNTU LINUX	Web server and agent host
APACHE2	Web server
MODSECURITY V2	Web Application Firewall
OWASP CRS	Attack detection rules
WAZUH AGENT	Log collection & forwarding
WAZUH MANAGER & DASHBOARD	SIEM monitoring & alerting
CURL	Attack simulation

Architecture Overview

Flow of Events:

1. Attacker sends malicious HTTP request
2. Apache receives the request
3. ModSecurity inspects the request using OWASP CRS
4. Malicious request is blocked (403)
5. ModSecurity logs are written to Apache error & audit logs
6. Wazuh Agent monitors these logs
7. Logs are sent to Wazuh Manager
8. Alerts are generated and visualized in Wazuh Dashboard

What is **ModSecurity** and Why you need it?



Implementation Steps

Step 1 – Apache Web Server Installation

Purpose:

To deploy a web server that will act as the protected application.

Action Performed:

- Installed Apache2 on Ubuntu

- Started and enabled the Apache service

```

● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-01 16:10:09 UTC; 45s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2948 (apache2)
    Tasks: 55 (limit: 1121)
   Memory: 5.2M (peak: 5.4M)
      CPU: 37ms
   CGroup: /system.slice/apache2.service
           └─2948 /usr/sbin/apache2 -k start
             └─2951 /usr/sbin/apache2 -k start
               └─2952 /usr/sbin/apache2 -k start

Feb 01 16:10:09 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Feb 01 16:10:09 apache2[2947]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using
Feb 01 16:10:09 ip systemd[1]: Started apache2.service - The Apache HTTP Server.

```

Step 2 – Apache Web Server Verification

Purpose:

To confirm that Apache is correctly installed and accessible.

Action Performed:

- Accessed server IP in the browser
- Verified Apache default “It works!” page



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```

/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf

```

Step 3 – ModSecurity Installation

Purpose:

To enable Web Application Firewall functionality on Apache.

Action Performed:

- Installed ModSecurity module for Apache
- Verified ModSecurity module is enabled

```
ubuntu@ip-10-10-10-10:~$ sudo apt install libapache2-mod-security2 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libapache2-mod-security2 is already the newest version (2.9.7-1build3).
0 upgraded, 0 newly installed, 0 to remove and 69 not upgraded.
ubuntu@ip-10-10-10-10:~$ sudo a2enmod security2
sudo systemctl restart apache2
Considering dependency unique_id for security2:
Module unique_id already enabled
Module security2 already enabled
ubuntu@ip-10-10-10-10:~$ apache2ctl -M | grep security
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.10.10.10. Set the 'ServerName' directive globally to suppress this message
security2_module (shared)
ubuntu@ip-10-10-10-10:~$
```

Step 4 – ModSecurity Configuration

Purpose:

To enable active inspection and request analysis.

Action Performed:

- Edited ModSecurity configuration file
- Changed rule engine mode to active blocking

```
# only to start with, because that minimises the chances of post-installation
# disruption.
#
SecRuleEngine On

# -- Request body handling -----
#
# Allow ModSecurity to access request bodies. If you don't, ModSecurity
# won't be able to see any POST parameters, which opens a large security
# hole for attackers to exploit.
#
SecRequestBodyAccess On
```

Step 5 – OWASP Core Rule Set (CRS) Installation

Purpose:

To load predefined security rules that detect common web attacks.

Action Performed:

- Installed modsecurity-crs
- Linked CRS rules to ModSecurity active rules directory
- Restarted Apache to apply rules

```

ubuntu@ip-10-10-10-10:~$ sudo apt install modsecurity-crs -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
modsecurity-crs is already the newest version (3.3.5-2).
modsecurity-crs set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 69 not upgraded.
ubuntu@ip-10-10-10-10:~$ sudo ln -s /usr/share/modsecurity-crs \
/etc/modsecurity/activated_rules
ubuntu@ip-10-10-10-10:~$ sudo systemctl restart apache2
ubuntu@ip-10-10-10-10:~$ ls /etc/modsecurity/activated_rules
owasp-crs.load  rules  util
ubuntu@ip-10-10-10-10:~$

```

Step 6 – Wazuh Agent Configuration for Log Monitoring

Purpose:

To forward ModSecurity security events to Wazuh SIEM.

Action Performed:

- Edited Wazuh Agent ossec.conf
- Added Apache error log and ModSecurity audit log for monitoring

```

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/error.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/modsec_audit.log</location>
</localfile>

```

Step 7 – Attack Simulation & Functional Testing

Purpose:

To verify that ModSecurity actively blocks malicious requests.

Action Performed:

- Sent SQL injection payload using curl
- Apache returned **403 Forbidden**, confirming attack blocking

```
ubuntu@ip-:~$ curl "http://:/?id=%27%20OR%201=1--"
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at : Port 80</address>
</body></html>
```

Step 8 – Raw Log Analysis on Server

Purpose:

To confirm ModSecurity is logging detailed attack information.

Action Performed:

- Reviewed Apache error logs
- Observed CRS rule IDs, severity, tags, and attack details

```
[Sun Feb 01 21:57:51.159225 2026] [security2:error] [pid 6724:tid 129770333398720] [client :55192] [client :55192] ModSecurity: Warning. Pattern match "^([\\d.]+)$" at REQUEST_HEADERS:Host. [file "/usr/share/modsecurity-crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line "736"] [id "920350"] [msg "Host header is a numeric IP address"] [data ""] [severity "WARNING"] [ver "OWASP CRS/3.3.5"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-protocol"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "capec/1000/210/272"] [tag "PCI/6.5.10"] [uri "/"] [unique_id "aX9_Bz2Inz3kR5o8emmsgQAAAAAM"]
[Sun Feb 01 21:57:51.159539 2026] [security2:error] [pid 6724:tid 129770333398720] [client :55192] [client :55192] ModSecurity: Warning. detected SQLi using libinjection with fingerprint 's6lc' [file "/usr/share/modsecurity-crs/rules/REQUEST-942-APPLICATION-ATTACK-SQLI.conf"] [line "66"] [id "942100"] [msg "SQL Injection Attack Detected via libinjection"] [data "Matched Data: s6lc found within ARGS:id: ' OR 1=1--"] [severity "CRITICAL"] [ver "OWASP CRS/3.3.5"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-sqli"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "capec/1000/152/248/66"] [tag "PCI/6.5.2"] [hostname ""] [uri "/"] [unique_id "aX9_Bz2Inz3kR5o8emmsgQAAAAAM"]
[Sun Feb 01 21:57:51.159658 2026] [security2:error] [pid 6724:tid 129770333398720] [client :55192] [client :55192] ModSecurity: Access denied with code 403 (phase 2). Operator GE matched 5 at TX:anomaly_score. [file "/usr/share/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "94"] [id "94910"] [msg "Inbound Anomaly Score Exceeded (Total Score: 8)"] [severity "CRITICAL"] [ver "OWASP CRS/3.3.5"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-generic"] [hostname ""] [uri "/"] [unique_id "aX9_Bz2Inz3kR5o8emmsgQAAAAAM"]
[Sun Feb 01 21:57:51.159811 2026] [security2:error] [pid 6724:tid 129770333398720] [client :55192] [client :55192] ModSecurity: Warning. Operator GE matched 5 at TX:inbound_anomaly_score. [file "/usr/share/modsecurity-crs/rules/RESPONSE-980-CORRELATION.conf"] [line "92"] [id "980130"] [msg "Inbound Anomaly Score Exceeded (Total Inbound Score: 8 - SQLI=5,XSS=0,RFI=0,LFI=0,RCE=0,PHPI=0,HTTP=0,SESS=0): individual paranoia level scores: 8, 0, 0, 0"] [ver "OWASP CRS/3.3.5"] [tag "event-correlation"] [hostname ""] [uri "/"] [unique_id "aX9_Bz2Inz3kR5o8emmsgQAAAAAM"]
```

Step 9 – Rule Loading Verification

Purpose:

To confirm OWASP CRS rules are properly loaded.

Action Performed:

- Observed CRS initialization logs
- Verified rule IDs and rule files being parsed

```
END-REQUEST-910-IP-REPUTATION"
/etc/modsecurity/activated_rules/rules/REQUEST-910-IP-REPUTATION.conf:SecRule TX:EXECUTING_PARANOIA_LEVEL "@lt 2" "id:910014,phase:2,pass,nolog,skipAfter:
END-REQUEST-910-IP-REPUTATION"
/etc/modsecurity/activated_rules/rules/REQUEST-910-IP-REPUTATION.conf:SecRule TX:EXECUTING_PARANOIA_LEVEL "@lt 3" "id:910015,phase:1,pass,nolog,skipAfter:
END-REQUEST-910-IP-REPUTATION"
/etc/modsecurity/activated_rules/rules/REQUEST-910-IP-REPUTATION.conf:SecRule TX:EXECUTING_PARANOIA_LEVEL "@lt 3" "id:910016,phase:2,pass,nolog,skipAfter:
END-REQUEST-910-IP-REPUTATION"
/etc/modsecurity/activated_rules/rules/REQUEST-910-IP-REPUTATION.conf:SecRule TX:EXECUTING_PARANOIA_LEVEL "@lt 4" "id:910017,phase:1,pass,nolog,skipAfter:
END-REQUEST-910-IP-REPUTATION"
/etc/modsecurity/activated_rules/rules/REQUEST-910-IP-REPUTATION.conf:SecRule TX:EXECUTING_PARANOIA_LEVEL "@lt 4" "id:910018,phase:2,pass,nolog,skipAfter:
END-REQUEST-910-IP-REPUTATION"
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004000,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004001,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004100,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004110,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004130,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004200,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004300,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004310,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004320,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004370,\
/etc/modsecurity/activated_rules/rules/REQUEST-903.9004-DOKUWIKI-EXCLUSION-RULES.conf: "id:9004380,\
```

Step 10 – Wazuh SIEM Alert Verification

Purpose:

To confirm centralized monitoring and alert generation.

Action Performed:

- Accessed Wazuh Dashboard
- Observed alerts with:
 - Rule description
 - Rule ID
 - Severity level
 - Source IP
 - MITRE ATT&CK mapping

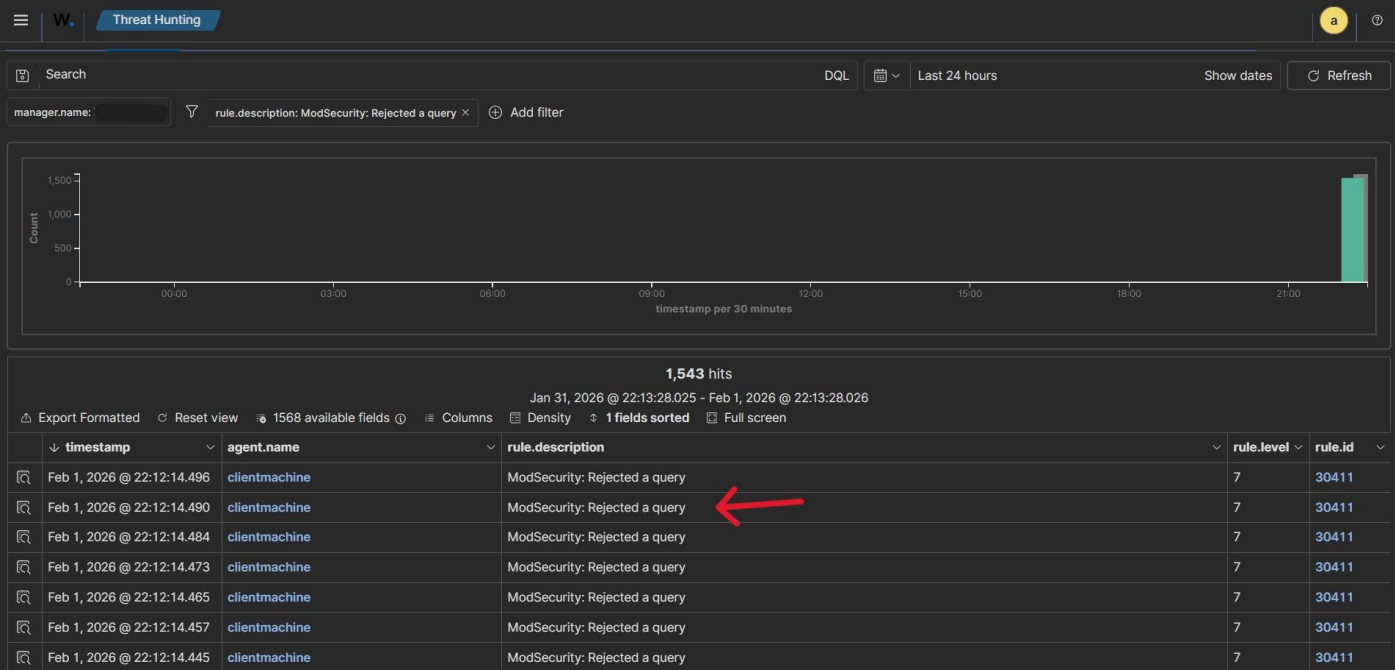


Table	JSON
r GeoLocation.city_name	Ashburn
r GeoLocation.country_name	United States
@ GeoLocation.location	{ "lon": -77.4728, "lat": 39.0481 }
r GeoLocation.region_name	Virginia
r _index	wazuh-alerts-4.x-2026.02.01
r agent.id	007
r agent.ip	
r agent.name	clientmachine
r data.id	ModSecurity
r data.srcip	
r decoder.name	apache-errorlog
r decoder.parent	apache-errorlog
r full_log	[Sun Feb 01 22:12:03.242540 2026] [security2:error] [pid 6724:tid 129769609881280] [client 32.123.456.78:48898] [client ""] Mod Security: Access denied with code 403 (phase 2). Operator GE matched 5 at TX:anomaly_score. [file "/usr/share/modsecurity-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "94"] [id "949110"] [msg "Inbound Anomaly Score Exceeded (Total Score: 5)"] [severity "CRITICAL"] [ver "OWASP_CRS/3.3.5"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-generic"] [hostname "ec2-32-123-456-78.compute-1.amazonaws.com"] [uri "/foo.php3"] [unique_id "aX-CWz2Inz3kR5o8emzGwAAABg"]
r id	1769964134.3713536
r input.type	log
r location	/var/log/apache2/error.log
r manager.name	
r rule.description	ModSecurity: Rejected a query
# rule.firedtimes	1,543
r rule.gpg13	10.1
r rule.groups	apache, web, modsecurity
r rule.id	30411
# rule.level	7
o rule.mail	false
r rule.mitre.id	T1083
r rule.mitre.tactic	Discovery
r rule.mitre.technique	File and Directory Discovery

Project Outcome

- Apache secured using ModSecurity
- OWASP CRS actively blocking attacks
- Web attacks detected in real time
- Logs centralized in Wazuh SIEM
- SOC-ready visibility of web threat

Skills Gained

- Web Application Firewall (WAF) deployment
- ModSecurity rule management
- OWASP CRS understanding
- SIEM log ingestion & alerting
- Web attack detection (SQLi, scanning)
- Apache security hardening
- SOC investigation workflows

Conclusion

This project demonstrates a **real-world Web Application Security Monitoring solution** by combining Apache, ModSecurity, OWASP CRS, and Wazuh SIEM.

It reflects how organizations protect web services against common attacks and how SOC teams gain visibility into malicious activity using centralized monitoring.

The setup closely mirrors enterprise-grade security architectures and provides strong hands-on experience in **WAF deployment, log analysis, and SIEM operations**.
